VILNIUS GEDIMINAS TECHNICAL UNIVERSITY

FACULTY OF ELECTRONICS

DEPARTMENT OF ELECTRNICS SYSTEMS

Machine Learning

Practical work 2

Authors: Kirill Postrehan, EDIfu 24/1

Lecturer: Dr. Olusola Oluwakemi Abayomi Alli

VILNIUS 2026

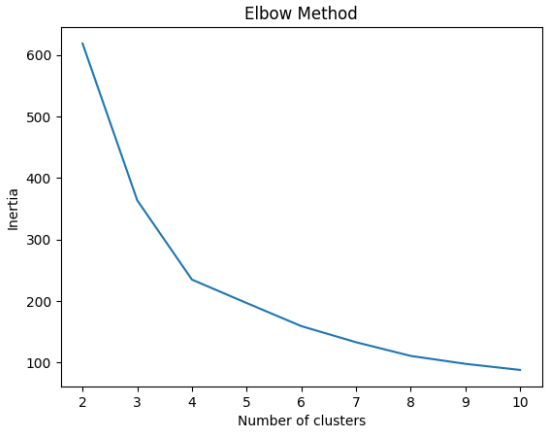
Introduction:

During this laboratory work I will use the dataset I got from scraped website with books and employ unsupervised machine learning algorithm K-Means to define “genres” for books based on their price and rating. Then I will try to classify books using Multi-Layer Perceptron and check metrics.

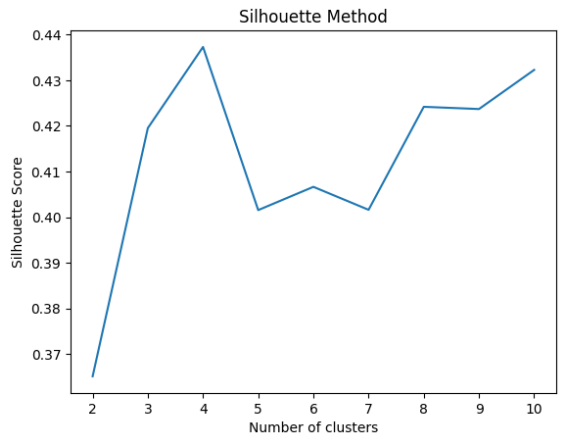
Analysis & visualizations:

I enerated a word cloud and it turned out that the highest keyword in the titles is “Love”.

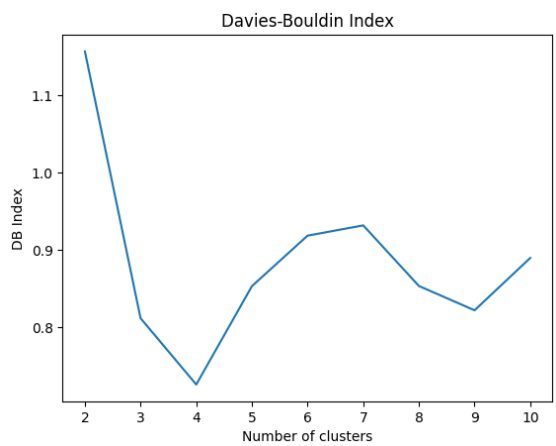
First thing first, I preprocessed the dataset in order to use it on K-Means and find the optimal number of clusters I need. I found out that the best option is 4 clusters based on inertia, silhouette and Davies-Bouldin index. Their graph clearly show us that.



(pic. 1. Inertia)



(pic. 2. silhouette)



(pic. 3. Davies-Bouldin Index)

All of them indicate that 4 clusters are the best.

I defined a dictionary of 4 random genres with corresponding cluster number and mapped them to get genre label.

The dataset of clustered books look as follows:

price	rating_num	Genre Lab	Genre
51.77	3	Horror	2
53.74	1	Horror	2
50.1	1	Horror	2
47.82	4	Horror	2
54.23	5	Horror	2
22.65	1	Detective	1
33.34	4	Sci-fi	0
17.93	3	Comedy	3
22.6	4	Detective	1
52.15	1	Horror	2

(pic. 4. dataset of clustered books)

Then, using torch I created class MLP with the following architecture:

```
class MLP(nn.Module):
  def __init__(self, in_features: int, hidden_size: int, out_features: int):
    super().__init__()
    self.mlp = nn.Sequential(
        nn.Linear(in_features=in_features, out_features=hidden_size),
        nn.ReLU(),
        torch.nn.Dropout(0.1),
        nn.Linear(in_features=hidden_size, out_features=hidden_size),
        nn.ReLU(),
        nn.Linear(in_features=hidden_size, out_features=out_features)
    )

  def forward(self, x):
    return self.mlp(x)
```

(pic. 5. the code for MLP class)

After finishing with preparing dataloaders and training function I started training the instance of MLP class. In order to reach roughly 95% accuracy and and 0.32 loss I experimented with different hyperparameters like number of epochs, hidden units, learning rate and its decay to find that 70 epochs, 20 hidden units and learning rate of 0.001 is what i needed.

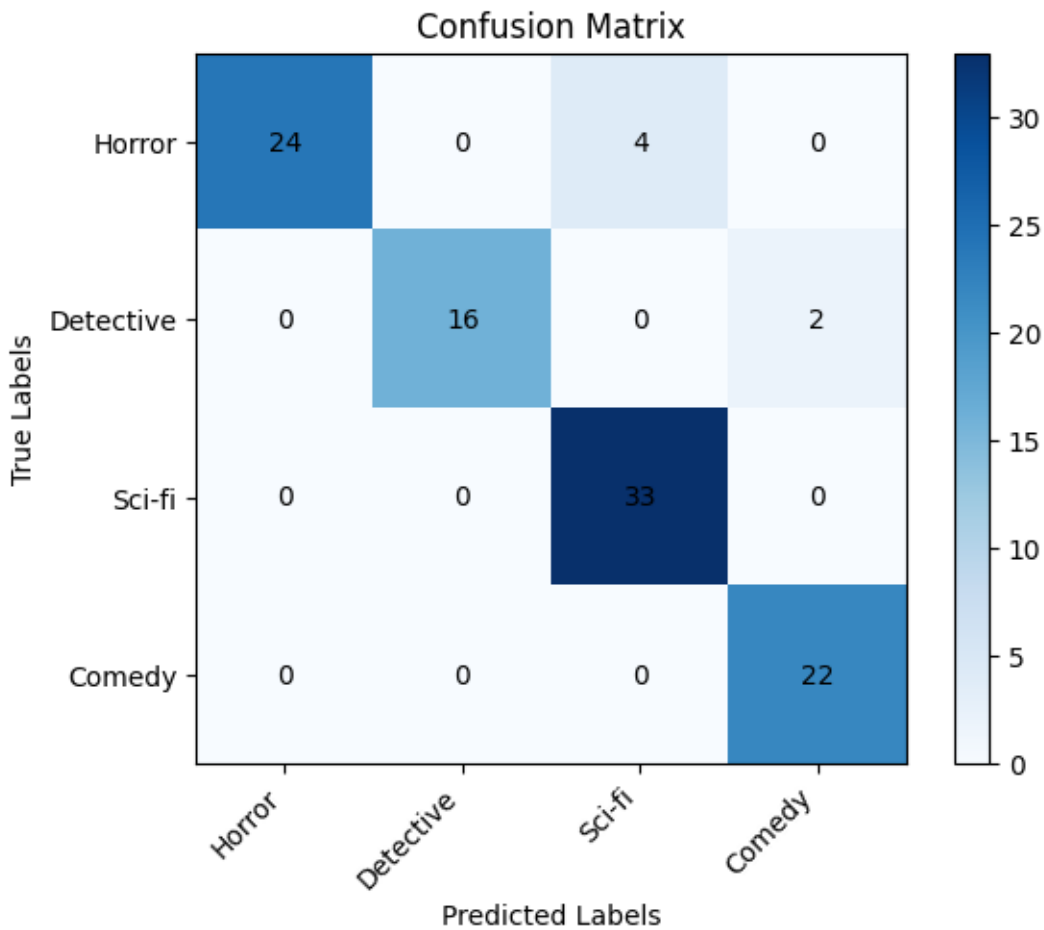
```
Training the model...: 100%|                                        70/70 [00:04<00:00, 18.17it/s]
Epoch: 10: Train loss: 1.1196 | train_acc: 0.41 | Test loss: 1.04 | Test acc: 0.5232
Epoch: 20: Train loss: 0.9068 | train_acc: 0.57 | Test loss: 0.86 | Test acc: 0.5589
Epoch: 30: Train loss: 0.7275 | train_acc: 0.67 | Test loss: 0.68 | Test acc: 0.6857
Epoch: 40: Train loss: 0.5633 | train_acc: 0.81 | Test loss: 0.56 | Test acc: 0.7661
Epoch: 50: Train loss: 0.5116 | train_acc: 0.80 | Test loss: 0.44 | Test acc: 0.8929
Epoch: 60: Train loss: 0.4308 | train_acc: 0.86 | Test loss: 0.38 | Test acc: 0.9196
Epoch: 70: Train loss: 0.3511 | train_acc: 0.89 | Test loss: 0.32 | Test acc: 0.9464
```

(pic. 6. The result of training)

Then I used metrics imported from torchmetrics library and got the following result:

```
Accuracy: 1.0000 | Precision: 1.0000 | Recall: 1.0000 | f1 score: 1.0000
```

(pic. 7. machine learning metrics)



(pic. 8. confusion matrix)

Conclusion:

In this practical work, K-Means clustering was used to identify four optimal clusters in the book dataset based on price and rating . Random genres were selected to represent these 4 clusters and later classified using a Multi-Layer Perceptron (MLP). After tuning hyperparameters, the model achieved about 95% accuracy with low loss, and evaluation metrics confirmed strong performance.